INTERVIEW STUDY GUIDE · VOLUME 0

Foundations for Understanding the Project

A plain-language guide to the build system, object code, static libraries, Kconfig, devicetree, out-of-tree development, and the meaning of “what we implemented and why.”

AM13E230 TINIElite Edge AI · Jainil Bavishi · November 2026

Contents

1. How to use this pack
2. From C source to firmware: object code and linking
3. Zephyr workspace and out-of-tree development
4. Kconfig, devicetree, and CMake — three different jobs
5. What “we implemented” actually means
6. The complete project flow
7. Interview answers to practise

1. How to use this pack

Read this volume first if the terms in the other documents feel dense. Then read Volume 1 for the project story, Volume 2 for the source-level design, Volume 3 for validation/history, and Volume 4 for interview practice.

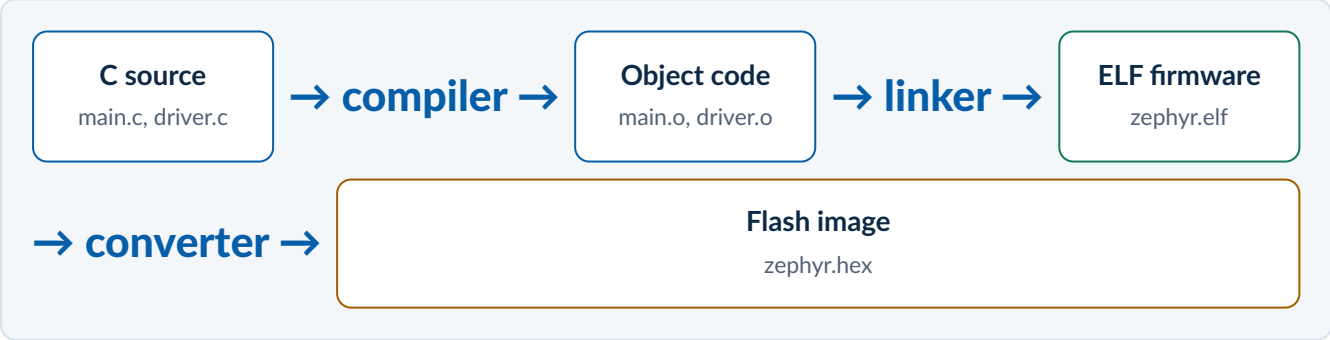
> **THE ONE-SENTENCE MENTAL MODEL**
> The project takes a supplied compiled AI model, builds it into firmware with the platform support that the unsupported board was missing, sends test data to the board safely, runs inference locally, and proves the important hardware behaviors with repeatable tests.

When a document says **"implemented"**, read it as **"designed, wrote, integrated, tested, or automated this surrounding platform/firmware behavior."** It does not mean that the neural network itself was trained or that the source model was created. The trained/generated model archive was supplied as an input.

2. From C source to firmware

2.1 Why source code cannot run directly

You write C or Python in human-readable source files. A Cortex-M33 processor does not execute C text. It executes machine instructions: binary bit patterns defined by its Arm instruction set. The build converts the source into those instructions and packs them with data at known memory addresses.



2.2 What is object code?

Object code is the compiler's binary translation of one source file. It is usually stored in an `.o` file. It contains machine instructions and data for that source file, but it is not normally a complete runnable program yet.

For example, `app_npu.c` can call a function defined elsewhere. Its object file records "I need a function with this name" rather than knowing where the function will finally live in flash. It can also contain unfinished address references. The **linker** resolves these references when it combines all object code into one final program.

Thing	Simple meaning	Example in this project

Source code	Text written by humans.	`app/src/app_npu.c`
Object file	Machine code made from one source file, with unresolved references allowed.	A build-directory `.o` compiled from `app_npu.c`
Static library / archive	A collection of object files placed in one file, normally ending in `.a`.	`mod.a`, the supplied generated model archive
Linker	Program that selects needed object code, resolves names, and assigns final addresses.	Links Zephyr, project drivers, and model archive into `zephyr.elf`
ELF	Debuggable executable format containing code, data, symbols, and memory layout.	`build/zephyr/zephyr.elf`
HEX	Text-based programming image describing bytes and flash addresses.	`build/zephyr/zephyr.hex` programmed by DSLite

2.3 What is mod.a?

`mod.a` is a static library: an archive of compiled object code produced by TI's model-generation tooling. It contains generated runtime/graph code and accelerator-related code, not readable C source for the model. During the link, the final firmware takes the necessary pieces from this archive and places them into `zephyr.elf`.

This is why this project is an **integration** project. The work was to give the supplied archive the environment it requires: exact symbols, memory callbacks, NPU register access, interrupts, clocks, input tensor preprocessing, and a way to test it on the board. It is not valid to say the archive was trained or regenerated here.

2.4 What does "model-package selectable" mean?

The original application named this one archive, its 256-byte input, its three-byte output, and its generated C structures directly. The current design moves those differences into `models/<name>/model.json`. The manifest declares tensor sizes and types plus the vendor-generated struct, field, entry, completion, and continuation names. CMake validates the package, runs `tools/generate_model_adapter.py`, compiles the resulting `build/generated/model_adapter.c`, and statically links the selected archive.

The generated adapter is ordinary C source, but nobody has to write it for each compatible package. It creates the vendor input/output structs, places the application buffers into the manifest-selected fields, calls the selected entry point, and exposes reset/finished/continue operations through the stable `app_model.h` API. This is **code generation**: structured metadata is converted into repetitive source code before the compiler runs.

This is **build-time model agnosticism**, not runtime loading. A different compatible model can be selected without rewriting the UART server or NPU wait/recovery logic, but firmware must still be rebuilt and flashed. An arbitrary `.a` cannot be sent over UART because it is target object code with linker symbols and relocations, not a self-describing model file. Generation removes repetitive hand-written glue; it cannot infer tensor meaning, preprocessing, labels, or an incompatible runtime ABI that the archive does not describe.

2.5 Why the linker matters in this project

A static archive is not automatically copied in full. Linkers usually pull only members that satisfy a needed symbol. The application link uses `-Wl,-u,tvmgen_default_run` so the model entry point is treated as needed. CI then inspects the final ELF symbols for the generated NPU kernels, the continuation handler, and project callback symbols. This is evidence that the intended binary pieces were linked, not merely stored in the repository.

> **USEFUL INTERVIEW DISTINCTION**
> "The archive is present" is weak evidence. "The final ELF contains its required symbols, and hardware reports its expected IRQ completions" is much stronger evidence of integration.

3. Zephyr workspace and out-of-tree development

3.1 What is a Zephyr workspace?

Zephyr projects normally use several Git repositories. A workspace is one parent directory containing the Zephyr source tree, this project, and required modules. The `west` tool reads `west.yml` to fetch the pinned revisions and to locate the workspace.

```
am13e230-workspace/
├── .venv/                          Python environment containing west
├── zephyr/                         pinned upstream Zephyr source
├── modules/hal/cmsis/              Arm CMSIS module
├── modules/hal/cmsis_6/            Arm CMSIS 6 module
└── am13e230-tinielite-zephyr/      this project repository
```

3.2 What "out of tree" means

Out of tree means project files live *outside* the upstream Zephyr source tree. "Tree" means the directory tree of a source repository. The project does not edit or paste files inside `zephyr/boards`, `zephyr/soc`, or `zephyr/drivers`. Instead, its own repository contains:

```
am13e230-tinielite-zephyr/
├── boards/       project board definition
├── soc/          AM13x SoC support and early clock code
├── dts/          hardware description and binding schemas
├── drivers/      project UART, pinctrl, and TINIElite drivers
├── app/          application firmware
└── zephyr/module.yml  declaration that lets Zephyr discover the above
```

During the build, Zephyr reads `zephyr/module.yml`. That metadata says, in effect, "also search this project for board, SoC, devicetree, Kconfig, and CMake content." Zephyr can then build a board that upstream Zephyr itself does not yet contain.

3.3 In-tree versus out-of-tree: concrete comparison

Approach	Where a new board driver would be placed	Problem or benefit
In-tree local modification	Inside a locally edited `zephyr/` checkout.	It may build on one laptop, but Git history is mixed with upstream, updates are hard, and a clean clone may miss invisible local edits.
Out-of-tree module (this project)	Inside this repository, discovered through `module.yml`.	The project repository holds its own changes. A clean pinned upstream checkout plus this repository can reproduce the build.
Upstreamed support	Accepted into the official Zephyr repository after review.	Best long-term reuse, but it needs reusable design, tests, documentation, and community review. This project has not reached that stage.

Out of tree does **not** mean "not part of the build." It means "part of the build without modifying upstream source." It also does not mean "temporary" or "wrong." It is a normal Zephyr way to develop a board and an application before deciding whether the reusable platform portion should be upstreamed.

3.4 Why it was especially important here

The inherited environment had an older Zephyr checkout with project edits and a duplicate board name. That made it easy for a build to accidentally depend on unstated local files. A clean out-of-tree repository plus a pinned manifest proves what this project actually needs, makes GitHub practical, and avoids destructively changing the legacy checkout.

4. Kconfig, devicetree, and CMake

These three systems are often confused because all act during a build. They answer different questions:

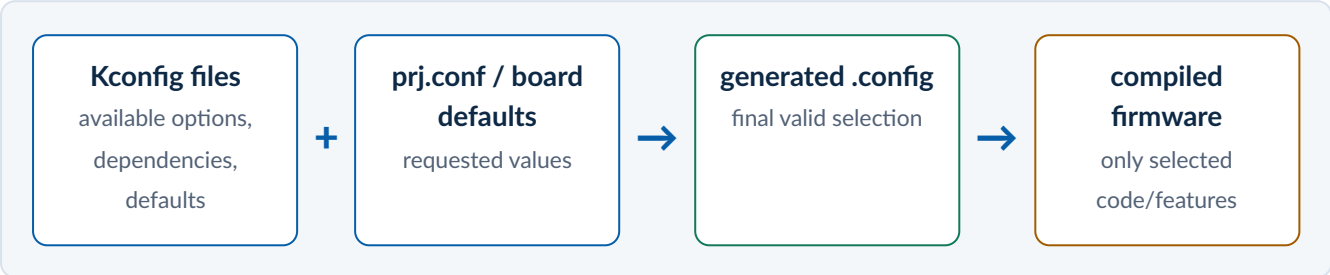
System	Question	Simple analogy	Project example
CMake	What source/object/library inputs are built and linked?	Which ingredients go into the recipe?	Compile application/driver C files and link `mod.a`.
Kconfig	Which software features and compile-time settings are selected?	Which optional recipe features are enabled?	Enable serial/reboot/NPU driver; set stack size; permit test-only fault injection.
Devicetree	Which physical hardware exists and how is it connected?	What kitchen equipment and wiring exist?	UART register blocks, IRQ 41, pins PA0/PA1, clock rates, NPU and watchdog addresses.

4.1 What is Kconfig, thoroughly?

Kconfig is Zephyr's compile-time configuration language and menu system. A Kconfig symbol normally starts with `CONFIG_` after configuration. It answers questions such as:

- Should a driver be compiled in?
- Should reboot support, serial support, or a kernel option be present?
- How large should the main thread stack be?
- Should a diagnostic-only fault command be accepted?

It is **compile time**, so it changes the built firmware image. It is not a value that a normal UART request can toggle after the firmware is flashed. To change a Kconfig option, edit configuration input, rebuild, and flash the new image.



For example, the project has test-only options:

```
CONFIG_APP_NPU_FAULT_INJECTION=y
CONFIG_APP_WATCHDOG_FAULT_INJECTION=y
```

With `=y`, diagnostic messages can intentionally create an NPU timeout or stop watchdog feeding so recovery can be proven. In a deployment configuration, set them to `n` or omit them so those commands do not exist in the firmware. This is why Kconfig is useful: it makes a feature explicitly selectable and lets dependencies prevent invalid combinations.

Typical Kconfig relationships are:

- **depends on:** feature B can only be enabled if A exists.
- **select:** enabling A automatically enables a required lower-level feature B.
- **default:** value used unless a more specific configuration overrides it.
- **choice:** exactly one option in a group may be selected.

4.2 Devicetree is not Kconfig

Devicetree describes a physical instance. For example, NPU base address `0x408ff000` and IRQ 41 are hardware facts. It would be wrong to treat them as a user preference. Kconfig decides whether the NPU driver feature is built; devicetree tells that driver which registers and interrupt it must use.

Similarly, Kconfig can enable serial support, but devicetree/pinctrl says which UART exists and which PA0/PA1 functions route its signals to pins. CMake then ensures the corresponding source files are compiled.

5. “What we implemented and why” — how to read it

The table in Volume 1 is not a list of random technologies. Each row has the same three-part engineering story:

1. **Implemented area:** the concrete subsystem added or changed.

2. **Why it was necessary:** the real gap or risk that existed without it.
3. **Key result:** the visible capability or evidence after the work.

Use this sentence pattern in an interview: **"The platform did not provide X, so I implemented Y. That let the system do Z, and I verified it by W."**

Problem before the work	What was implemented	Why it matters now
Zephyr did not know this preview board existed.	Out-of-tree board, SoC, devicetree, pinctrl, and driver support.	Firmware can build reproducibly for the real LP-AM13E230 instead of relying on private edits.
The real clock was not configured to match assumed timing.	Crystal/PLL/flash initialization plus register telemetry.	UART settings, flash safety, and measured microseconds are tied to actual 200 MHz hardware.
Raw serial bytes could be damaged, truncated, or stale.	Versioned UART frames with length, CRC-32, sequence, errors, deadline, and resync.	A test client can identify the correct response and recover rather than silently accepting bad data.
Common firmware was coupled to one generated model ABI.	Validated model manifests, stable adapter API, archive-derived identity, and protocol-v3 tensor metadata.	Compatible packages can be selected at build time, while host/board mismatches fail before inference.
The supplied model archive needed hardware-specific functions.	TINIElite callbacks, NPU power/privilege setup, SRAM staging, IRQ handling.	The generated runtime can use the real accelerator, not just be stored as a file.
A missing NPU completion could hang the application.	100 ms bounded wait, structured error, cold reset.	Known accelerator failures become observable/recoverable instead of infinite waits.
A total CPU/application hang cannot execute software timeout logic.	Independently clocked WWDT0, fed only by main-loop progress.	The board can reset even when normal firmware control flow is broken.
Manual, one-off testing would be hard to trust or repeat.	Python regression client, scripts, tests, CI, JSON hardware evidence.	Another person can repeat build/flash/test and see exactly what was measured.

5.1 What was supplied versus what was our engineering work

Supplied input (do not overclaim)	Project engineering work
Generated model archive `mod.a`, header, licence, and six golden fixtures.	Linked archive into firmware; supplied ABI callbacks, input preparation, target drivers, interrupt path, and validation.
TI chip/board, SDK register information, XDS110 debug probe, and TI flashing tool.	Created Zephyr board/SoC/DTS integration; configured clock/UART/NPU/watchdog; made WSL workflow reproducible.
Zephyr RTOS and CMSIS upstream dependencies.	Pinned dependencies, integrated as a module, wrote app/driver code, tests, scripts, docs, and CI checks.

6. Complete flow from typing a command to seeing a result

1. You run `./scripts/build.sh`.
2. west finds pinned Zephyr/modules and discovers this out-of-tree project through `module.yml`.
3. CMake validates the selected `model.json`, selects sources, adapter, header, and archive.
4. Kconfig resolves which features are compiled; devicetree generates the hardware constants.
5. The Arm compiler produces object code; the linker makes `zephyr.elf`; conversion makes `zephyr.hex`.
6. `./scripts/flash.sh` uses Linux DSLite and XDS110 to erase, program, and verify the board.
7. On boot, firmware safely configures the 200 MHz clock, drivers, NPU, UART, and watchdog, then sends READY.
8. The Python client quantizes a 256-sample fixture, sends a CRC-protected frame, and receives ACK.
9. The generated runtime runs CPU and NPU stages; DONE IRQ 41 wakes the thread for each accelerator completion.
10. The board returns scores/timing/IRQ counts. The host checks CRC, sequence, exact scores, nonzero completions, and zero spurious IRQs.

7. Short answers to practise

What is object code?

Object code is the binary machine-code translation of one source file. It can still contain unresolved references. The linker combines object code and libraries into the final ELF firmware.

What is a static library?

It is an archive of object code, usually a .a file. Here mod.a is a supplied generated model archive. At link time the required members become part of the final firmware, so there is no dynamic loading on the board.

Is the project model agnostic?

It is build-time selectable across compatible AM13E2/TINIElite packages. A manifest supplies tensor and artifact metadata, and a small adapter hides generated names. It is not a runtime loader: each archive must be compatible, validated, linked into a new ELF, flashed, and tested.

What does out-of-tree mean here?

The AM13E230 board, SoC, devicetree, and drivers live in our own repository, not inside upstream Zephyr. module.yml lets a clean pinned Zephyr checkout discover them during build. This keeps changes reproducible and separate from upstream.

What is Kconfig?

Kconfig is Zephyr's compile-time feature-selection system. It decides what software support is compiled into a specific firmware image—for example drivers, stack sizes, and test-only fault interfaces. It is different from devicetree, which describes physical hardware, and CMake, which selects sources/libraries to build.

What did you implement, as opposed to what was given?

The model archive, header, and six fixtures were supplied. I implemented the Zephyr platform support and firmware environment around them: board/SoC/DTS integration, clock/pins/UART/NPU/watchdog support, framed host protocol, recovery, WSL flashing/testing, and validation/CI. I did not train or regenerate the model.

Next: read Volume 1 for the project story and the complete implementation map.